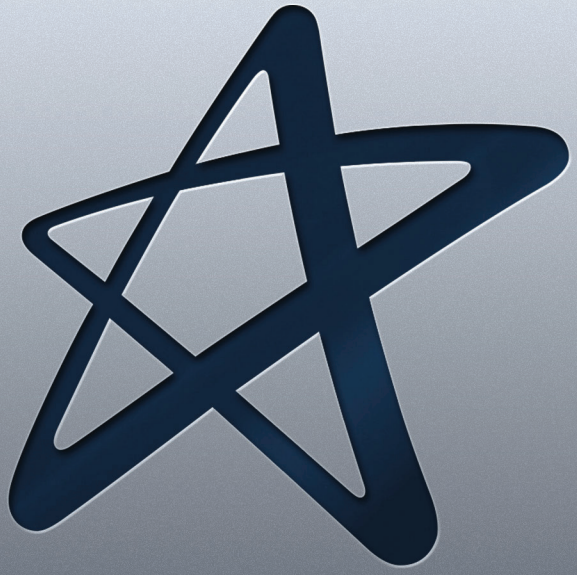
Programação para Dispositivos Móveis



Cruzeiro do Sul Virtual
Educação a Distância

Material Teórico

View, eventos, activity e intent

Responsável pelo Conteúdo:
Prof. Ms. Luis Carlos Reis

Revisão Textual:
Prof. Ms. Douglas Almendro

Revisão Textual:
Profa. Ms. Luciene Oliveira da Costa Santos

UNIDADE View, eventos, activity e intent



- Introdução
- View e eventos
- Objetos de tipo RadioButton e CheckBox
- Interação entre telas
- Activity
- Intent
- Utilizando *Activity* sem esperar resultados
- Configuração necessária no arquivo AndroidManifest.xml
- Explicação

OBJETIVO DE APRENDIZADO

- Demonstrado como a aplicação pode interagir com o usuário;
- Analisar as interações entre as telas da aplicação;

ORIENTAÇÕES

Olá, aluno(a)!

Estamos começando nossos estudos sobre desenvolvimento mobile mais especificamente estamos falando de Android.

Com o conhecimento adquirido com os conceitos iniciais de layout e algumas clássicas básicas a proposta desta unidade é o estudo de eventos e interações entre a aplicação e o usuário.

Não deixe de assistir, também, à apresentação narrada do conteúdo e de alguns exercícios resolvidos.

Finalmente, e o mais importante, fique atento às atividades avaliativas propostas e ao prazo de realização e envio.

Bom estudo!

Contextualização

Ao desenvolver uma aplicação, devemos estar familiarizados com alguns conceitos básicos relacionados à sua estrutura. Entre eles, podemos citar dois componentes essenciais para toda aplicação: as activities e os intents.

Uma activity é uma classe Java que herda da classe activity do pacote Android. Ela define a tela que irá interagir com um usuário. Para desenvolver qualquer aplicação com interface em Android é necessário que se tenha pelo menos uma activity.

A activity é apenas a tela exibida, sem nenhum layout, devemos utilizar os arquivos xml de layout, nos quais se definem todos os elementos visuais, tais como botões e imagens.

Nos botões e imagens, podemos adicionar os eventos para criar a interação na aplicação e, assim, iremos construir um aplicativo.

Introdução

Os aplicativos necessitam, frequentemente, interagir com os usuários e, em forma geral, atender situações que aconteçam eventos, executando ações definidas para esta intenção.

Nesta aula, aprenderemos a utilizar os eventos, para termos maior interação com o usuário. Iremos aprender, também, a interação entre telas e, para isso, iremos utilizar os conceitos de *activity* e *intent* com o intuito de navegar entre as telas e transferir conteúdos entre as mesmas.

View e eventos

A programação para Android, semelhantemente a outros ambientes, linguagens e ferramentas gráficas, é orientada a eventos, neste caso, aos cliques e toques na tela.

Cada vez que um usuário clica em um botão, seleciona um item em uma lista, ou pressiona uma tecla, o sistema operacional gera um evento. Se uma aplicação está interessada em um evento específico (por exemplo, clique em um botão), deve solicitar ao sistema para "escutar" o evento. Se a aplicação não está interessada, seu processamento continua de forma normal.

É importante observar que a aplicação não espera pela ocorrência de eventos isso é controlado pelo sistema. Para que um componente ou container possa "escutar" eventos, é preciso instalar um *listener*.

Listeners são classes criadas, especificamente, para o tratamento de eventos. Um *event listener* é uma interface da classe *View* que contém um método simples de chamada. Esse método pode ser chamado pela *framework* Android, quando a *View*, a qual o *listener* está registrado, é chamada por uma interação de usuário com um item da interface, ou seja, quando a ação correspondente ocorre no objeto.

Por exemplo, quando um botão é clicado, o método *onClick*() é chamado no objeto. Os principais eventos presentes na programação para Android são:

onClick()

Vem de *View.OnClickListener*. É chamado quando o usuário toca o item (quando estiver em modo de toque) ou foca o item através de teclas de navegação, ou *trackball* e pressiona o botão de enter correspondente, ou pressiona o *trackbak* (que também serve como enter).

onLongClick()

Vem de *View.OnLongClickListener*. É chamado quando o usuário toca um item e o segura (quando estiver em modo de toque) ou foca o item através de teclas de navegação ou *trackball* e pressiona o botão de *enter* correspondente e o segura ou pressiona o *trackbak* por, pelo menos, um segundo.

onFocusChange()

Vem de *View.OnFocusChangeListener*. É chamado quando o usuário navega para dentro ou para fora de um item, usando as teclas de navegação ou *trackball*.

onKey()

Vem de *View.OnKeyListener*. É chamado quando o usuário foca em um item e pressiona ou solta uma tecla no dispositivo. Exemplo: quando se está escrevendo uma mensagem, você pode tocar no botão virtual (ou físico) da letra A e, ao soltá-lo, a letra A é mostrada na tela.

onTouch()

Vem de *View.OnTouchListener*. É chamado quando o usuário performa uma ação qualificada como um evento de toque, incluindo pressionar, soltar ou qualquer movimento de gesto na tela (dentro dos limites do item).

onCreateContextMenu()

Vem de *View.OnCreateContextMenuListener*. É chamado quando um menu de contexto está sendo criado (como resultado de um long click).

Para exemplificar os *listeners* relacionados aos eventos acima, vamos criar um projeto novo de calculadora com nome Eventos.



Figura 1

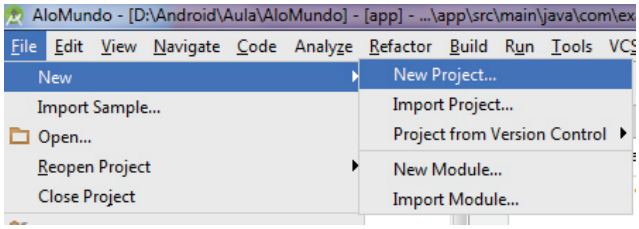


Figura 2

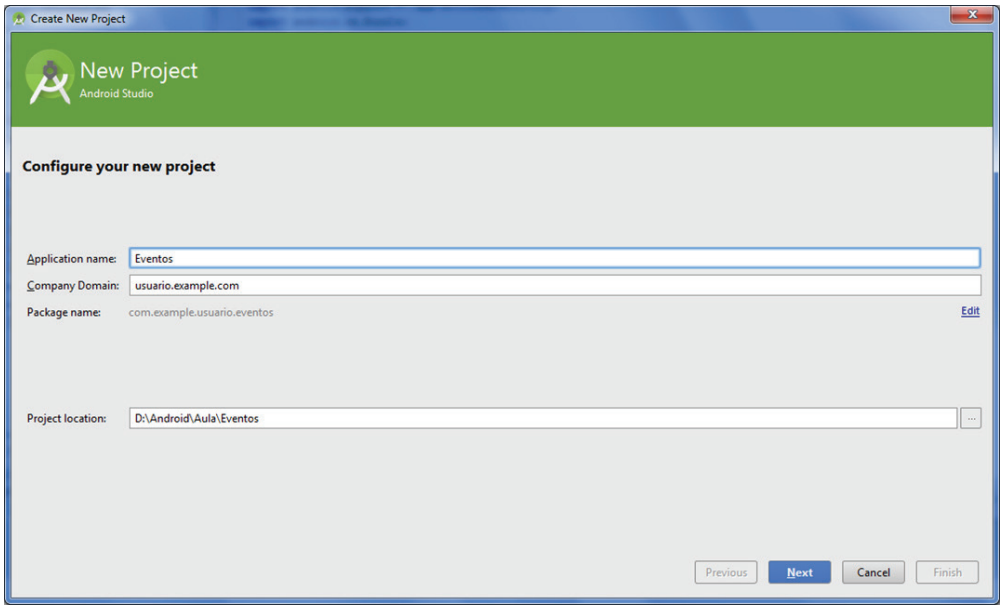


Figura 3

Na próxima tela, selecionar *Finish*.

Após a criação, crie um layout com nome **eventoslayout.xml** conforme abaixo:

Botão da direita em *layout*, new, *Android XML File*

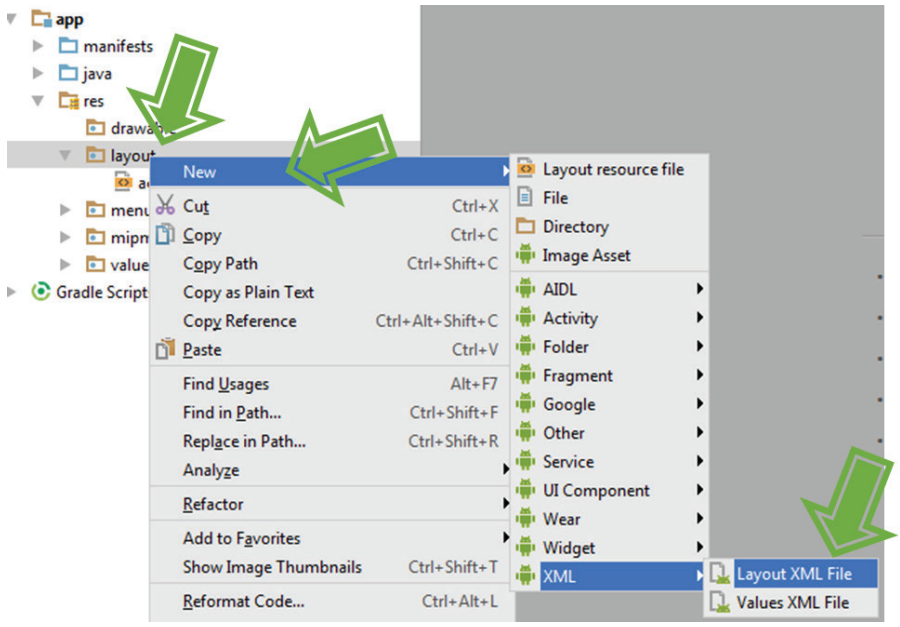


Figura 4

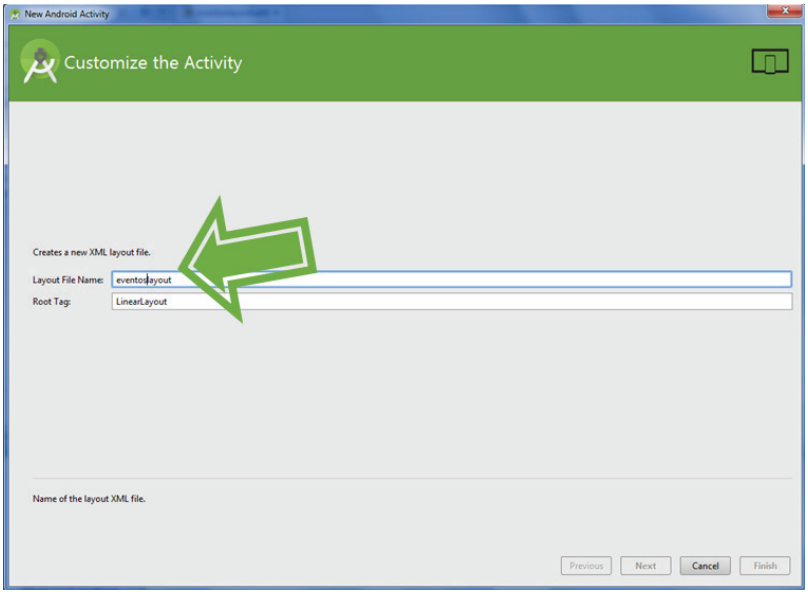


Figura 5

Criado o projeto, o primeiro passo será alterar o XML do layout. Nele, precisaremos de:

- Um campo EditText para receber o valor "1";
- Um campo EditText para receber o valor "2";
- Um campo Button para o botão "Somar";
- Um campo Button para o botão "Subtrair";
- Um campo Button para o botão "Multiplicar";
- Um campo Button para o botão "Dividir";
- Um campo EditText para exibir o resultado.

Sendo assim, substitua o código do layout gerado por esse abaixo, onde será responsável pela geração da imagem abaixo.



Figura 6

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/
res/android"
    android:layout_width="match_parent" android:layout_
height="match_parent">
    <LinearLayout android:orientation="vertical"
android:layout_width="fill_parent" android:layout_height="fill_
parent">
        <TextView android:layout_width="fill_parent"
android:layout_height="wrap_content" android:text="Digite o
1º numero: "/>
        <EditText android:layout_width="fill_parent"
android:layout_height="wrap_content" android:id="@+id/num1"
android:numeric="decimal"/>
        <TextView android:layout_width="fill_parent"
android:layout_height="wrap_content" android:text="Digite o
2º numero: "/>
        <EditText android:layout_width="fill_parent"
android:layout_height="wrap_content" android:id="@+id/num2"
android:numeric="decimal"/>
        <LinearLayout android:orientation="horizont
al" android:layout_width="wrap_content" android:layout_
height="wrap_content">
            <Button android:layout_width="wrap_content"
android:layout_height="wrap_content" android:id="@+id/btSo-
mar" android:text="Somar"/>
            <Button android:layout_width="wrap_content"
android:layout_height="wrap_content" android:id="@+id/btSub-
trair" android:text="Subtrair"/>
            <Button android:layout_width="wrap_content"
android:layout_height="wrap_content" android:id="@+id/btDi-
vidir" android:text="Dividir"/>
            <Button android:layout_width="wrap_content"
android:layout_height="wrap_content" android:id="@+id/btMul-
tiplicar" android:text="Multiplicar"/>
        </LinearLayout>
        <TextView android:layout_width="fill_parent"
android:layout_height="wrap_content" android:text="Resultado:
"/>
        <EditText android:layout_width="fill_parent"
android:layout_height="wrap_content" android:id="@+id/re-
sult"/>
    </LinearLayout>

</LinearLayout>
```

Obs.: Control+Alt+L alinha o texto acima



Importante!

Notem que selecionando cada objetos (botão, campo texto, etc), no item ID, se refere ao nome de cada objeto, no qual utilizaremos na programação posteriormente. Na janela de propriedades, vejam no item Text o valor que aparece em cada botão.

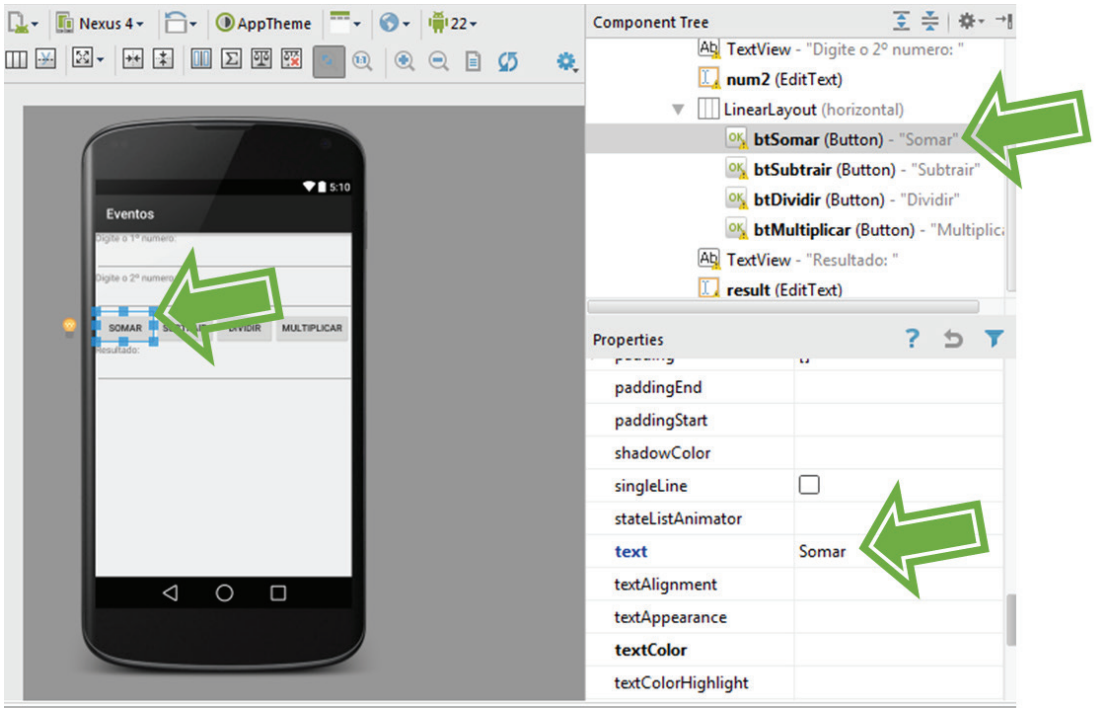


Figura 7

Na classe principal (*MainActivity*.java, consulte a janela *Project*), acrescente às cláusulas *import* os seguintes itens:

```
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
```

Portanto sua classe conterá:

```
import android.support.v7.app.ActionBarActivity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;

import android.view.View;
import android.widget.Button;
import android.widget.EditText;
```

Na classe *MainActivity*, iremos criar variáveis que serão referenciados aos objetos criados na classe:

```
public class MainActivity extends ActionBarActivity {

    //Campos do layout
    EditText num1, num2, result;
    Button btSomar, btSubtrair, btDividir, btMultiplicar;

    //Vars da classe para o calculo
    double n1, n2, resultado;
```

Altere o método *onCreate* para que aponte para o novo layout.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.eventoslayout);
}
```

Altere o método *onCreate* para que referencie aos objetos do novo layout e criar os métodos no qual o evento será acionado para cada botão.

```
public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.eventoslayout);

//Pegando os valores do usuario
num1    = (EditText) findViewById(R.id.num1);
num2    = (EditText) findViewById(R.id.num2);
result  = (EditText) findViewById(R.id.result);

btSomar       = (Button)   findViewById(R.id.btSomar);
btSubtrair    = (Button)   findViewById(R.id.btSubtrair);
btDividir     = (Button)   findViewById(R.id.btDividir);
btMultiplicar = (Button)   findViewById(R.id.btMultiplicar);


//BOTAO SOMAR
btSomar.setOnClickListener(new View.OnClickListener() {

    public void onClick(View v) {
        //Passando os valores do layout para as vars da classe
        n1 = Double.parseDouble(num1.getText().toString());
        n2 = Double.parseDouble(num2.getText().toString());

        resultado = n1+n2;

        //Passando os valores da soma para o campo Resultado
do layout
       result.setText(String.valueOf(resultado));
    }
});

//BOTAO SUBTRAIR
btSubtrair.setOnClickListener(new View.OnClickListener() {

    public void onClick(View v) {
        //Passando os valores do layout para as vars da classe
        n1 = Double.parseDouble(num1.getText().toString());
        n2 = Double.parseDouble(num2.getText().toString());

        resultado = n1-n2;
```

```
            //Passando os valores da soma para o campo Resultado
    do layout

            result.setText(String.valueOf(resultado));
        }
    });

    //BOTAO DIVIDIR
    btDividir.setOnClickListener(new View.OnClickListener() {

        public void onClick(View v) {
            //Passando os valores do layout para as vars da classe
            n1 = Double.parseDouble(num1.getText().toString());
            n2 = Double.parseDouble(num2.getText().toString());

            resultado = n1/n2;

            //Passando os valores da soma para o campo Resultado
    do layout
            result.setText(String.valueOf(resultado));
        }
    });

    //BOTAO MULTIPLICAR
    btMultiplicar.setOnClickListener(new View.OnClickListener() {

        public void onClick(View v) {
            //Passando os valores do layout para as vars da classe
            n1 = Double.parseDouble(num1.getText().toString());
            n2 = Double.parseDouble(num2.getText().toString());

            resultado = n1*n2;

            //Passando os valores da soma para o campo Resultado
    do layout
            result.setText(String.valueOf(resultado));
        }
    });
    }
```

Uma vez já configurado o emulador, será somente necessário executar o projeto.



Figura 8

Analisando a programação do exemplo anterior

```
public class MainActivity extends ActionBarActivity {

    //Campos do layout
    EditText num1, num2, result;
    Button btSomar, btSubtrair, btDividir, btMultiplicar;

    //Vars da classe para o calculo
    double n1, n2, resultado;
```

Primeiramente, declaramos as variáveis no qual utilizaremos posteriormente no decorrer da classe.

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.eventoslayout);

        //Pegando os valores do usuario
        num1    = (EditText) findViewById(R.id.num1);
        num2    = (EditText) findViewById(R.id.num2);
        result  = (EditText) findViewById(R.id.result);

        btSomar       = (Button)   findViewById(R.id.btSomar);
        btSubtrair    = (Button)   findViewById(R.
id.btSubtrair);
        btDividir     = (Button)   findViewById(R.
id.btDividir);
        btMultiplicar = (Button)   findViewById(R.
id.btMultiplicar);
```

O método onCreate será chamado na primeira fase do ciclo de vida da Activity, quando esta for criada.



Disponível em: https://goo.gl/FqQey

O objeto *savedInstanceState* permite referenciar um estado anterior da instância (se foi armazenado antes de entrar em pausa). No segundo comando, decidimos o conteúdo que será mostrado na *Activity*, de acordo com o projeto visual no arquivo **eventoslayout.xml**. O termo R.layout. **eventoslayout** especifica que, no arquivo R.java, gerado automaticamente pelo Android Studio, existe uma classe *layout*, com a declaração de uma constante *main* e ela estará relacionada com um arquivo existente **eventoslayout.xml**.

Nas próximas linhas, pegamos as referências de vários objetos ou componentes do visual utilizado. Por exemplo, `num1=(EditText) findViewById(R.id.num1)` nos permite acessar o valor digitado pelo usuário no objeto de entrada de texto identificado por R.id.**num1** (constante também gerada dentro de R.java e referenciada dentro do *layout*).

Para obter o valor, executaremos posteriormente:

```
        n1 = Double.parseDouble(num1.getText().toString());

    //BOTAO SOMAR
    btSomar.setOnClickListener(new View.OnClickListener() {

        public void onClick(View v) {
            //Passando os valores do layout para as vars da classe
            n1 = Double.parseDouble(num1.getText().toString());
            n2 = Double.parseDouble(num2.getText().toString());

            resultado = n1+n2;

            //Passando os valores da soma para o campo Resultado
    do layout
            result.setText(String.valueOf(resultado));
        }
    });
```

Para especificar que queremos atender o evento de *click* do objeto btSomar (botão, objeto da classe *Button*) executamos o método setOnClickListener. Dessa forma, o método onClick, que programamos dentro, será chamado quando esse evento aconteça.

A lógica mostrada obtém as strings digitadas pelo usuário nos objetos num1 e num2, converte-as em valores reais e mostra o resultado da soma destes valores no na variável resultado, no qual, posteriormente, será atribuído ao objeto result (objeto da classe *EdiText*). Observe a utilização dos métodos *getText() e setText().*

Outros eventos

Obviamente, existem outros tipos de eventos. Por exemplo, Android prevê atender o evento de click longo: **onLongClick**, bastante útil em aparelhos com tela sensível ou *touchscreen*. Frequentes, também, os eventos: **onKey** (usar *setOnKeyListener*) e **onTouch** (usar *setOnTouchListener*).

```
    public class Eventos extends Activity {
        ...
       public void onCreate(Bundle savedInstanceState) {
            ...
```

```
        final Button botao = (Button) findViewById(R.id.btn-
mult);
        botao.setOnLongClickListener (new View.OnLongClick-
Listener() {
            public boolean onLongClick(View v) {
                Toast.makeText(Eventos.this, "\nMuito bem.
Evento click longo atendido...\n", Toast.LENGTH_LONG).show();
                return true;
            }
        });
    }
    ...
}
```

Objetos de tipo RadioButton e CheckBox

O *RadioButton* serve para questões de escolha única, enquanto o *CheckBox* é utilizado para múltiplas escolhas.

Para maiores informações sobre esses objetos, consulte a documentação em:
https://goo.gl/CViGFq

Iremos elaborar um novo projeto no qual abordaremos esses dois objetos e também a navegação entre telas. Para isso, crie um projeto novo com o nome de "Cadastro". No menu *"File – New – New Project"*

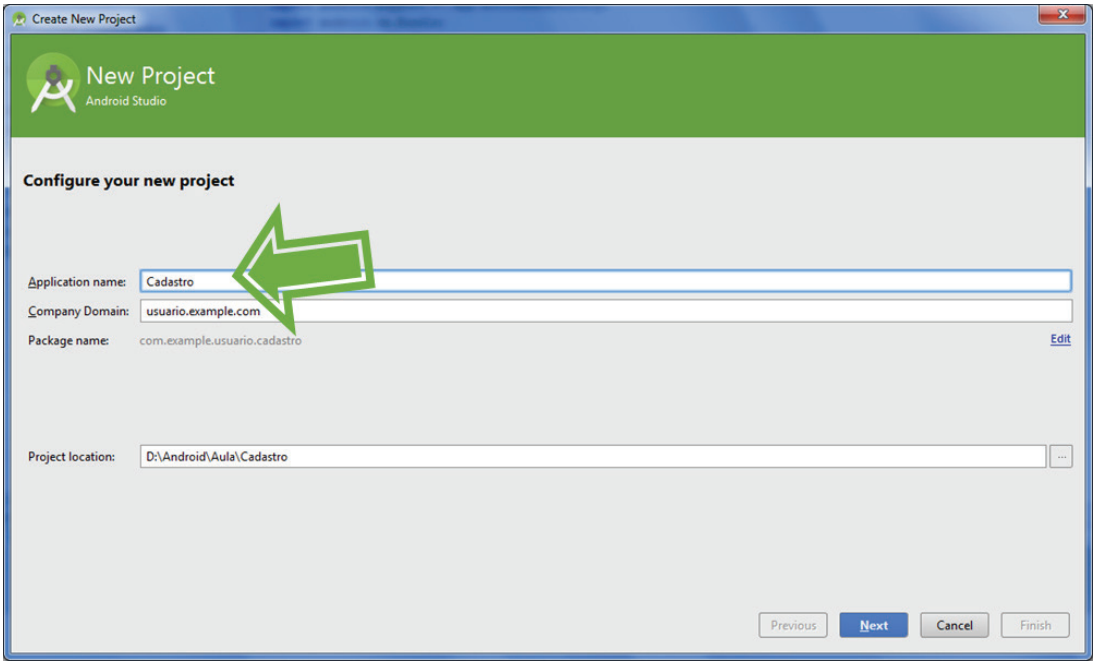


Figura 9

Conforme visto anteriormente, crie um novo *layout* com o nome "cadastro", e altere o *layout* conforme abaixo:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/
res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TableLayout
        android:layout_width="fill_parent"
        android:layout_height="fill_parent"
        android:stretchColumns="1">
        <TableRow>
            <TextView android:text="CPF:"
                android:textStyle="normal|bold"
                android:id="@+id/lblcpf"
                android:padding="3dip"
                android:gravity="left" />
        </TableRow>
        <TableRow>
            <EditText android:text=""
                android:id="@+id/txtcpf"
                android:gravity="left"
                android:padding="5dip"
                android:maxLength="14"
                android:width="25dp"
                android:layout_span="2" />
        </TableRow>

        <TableRow>
            <TextView android:text="Nome:"
                android:textStyle="normal|bold"
                android:id="@+id/lblnome"
                android:padding="3dip"
                android:gravity="left" />
        </TableRow>
        <TableRow>
            <EditText android:text=""
                android:id="@+id/txtnome"
                android:gravity="left"
                android:padding="5dip"
                android:layout_span="2" />
        </TableRow>
        <TableRow>
            <TextView android:text="Idade:"
                android:textStyle="normal|bold"
                android:id="@+id/lblidade"
                android:padding="3dip"
                android:gravity="left" />
    </TableRow>
```

```
        <TableRow>
            <EditText android:text=””
                android:id=”@+id/txtidade”
                android:gravity=”left”
                android:padding=”5dip” />
        </TableRow>

        <TableRow>
            <TextView
                android:id=”@+id/TextLinha01”
                android:layout_width=”wrap_content”
                android:layout_height=”wrap_content”
                android:layout_span=”2” />
        </TableRow>

        <TableRow>
            <TextView
                android:id=”@+id/TextView01”
                android:layout_width=”wrap_content”
                android:layout_height=”wrap_content”
                android:layout_above=”@+id/chkStar”
                android:layout_centerHorizontal=”true”
                android:text=”Selecione seus filmes de inter-
esses”
                android:textAppearance=”?android:attr/textAp-
pearanceLarge”
                android:layout_span=”2” />
        </TableRow>

        <TableRow>
            <CheckBox
                android:id=”@+id/chkSenhor”
                android:layout_width=”wrap_content”
                android:layout_height=”wrap_content”
                android:layout_above=”@+id/btnDisplay”
                android:text=”Senhor dos Anéis” />
        </TableRow>

        <TableRow>
            <CheckBox
                android:id=”@+id/chkGame”
                android:layout_width=”wrap_content”
                android:layout_height=”wrap_content”
                android:layout_above=”@+id/chkSenhor”
                android:layout_alignLeft=”@+id/chkSenhor”
                android:text=”Game of Thrones” />
        </TableRow>

        <TableRow>
            <CheckBox
                android:id=”@+id/chkStar”
                android:layout_width=”wrap_content”
```

```
                android:layout_height="wrap_content"
                android:layout_above="@+id/chkGame"
                android:layout_alignLeft="@+id/chkGame"
                android:text="Star Wars" />
        </TableRow>

        <TableRow>
            <TextView
                android:id="@+id/TextLinha02"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_span="2" />
        </TableRow>

        <TableRow>
            <TextView
                android:id="@+id/textView1"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_centerHorizontal="true"
                android:text="Selecione seu sexo"
                android:textAppearance="?android:attr/textAp-
pearanceLarge"
                android:layout_span="2"/>
        </TableRow>


        <RadioGroup
            android:id="@+id/radioSex"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_below="@+id/textView1"
            android:layout_centerHorizontal="true" >

            <RadioButton
                android:id="@+id/radioM"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Masculino" />

            <RadioButton
                android:id="@+id/radioF"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Feminino" />
        </RadioGroup>


        <TableRow>
            <Button
               android:id="@+id/btnDisplay"
                android:layout_width="wrap_content"
```

```
                android:layout_height="wrap_content"
                android:layout_alignParentBottom="true"
                android:layout_centerHorizontal="true"
                android:text="Mostrar em Outra Tela" />
        </TableRow>
    </TableLayout>
</LinearLayout>
```

Com isso, seu *layout* ficará conforme figura abaixo:



Figura 10

Na programação, acesse a classe *MainActivity* e importe os seguintes objetos:

```
import android.support.v7.app.ActionBarActivity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;

import android.content.Intent;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.RadioButton;
import android.widget.RadioGroup;
```

Logo abaixo do "*public class MainActivity*", declare as variáveis de cada objeto que iremos utilizar.

```
public class MainActivity extends ActionBarActivity {

    private RadioGroup radioSexGroup;
    private RadioButton radioSexButton;
    private Button btnDisplay;
    private String cpf, nome, idade, sexo;
    private CheckBox chkStar, chkGame, chkSenhor;
    private Intent intent;
    private EditText editCpf, editNome, editIdade;
```

No método *onCreate*, iremos mudar para o novo *layout* "cadastro" `setContentView(R.layout.cadastro);`

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.cadastro);

    editCpf = (EditText) findViewById(R.id.txtcpf);
    editNome = (EditText) findViewById(R.id.txtnome);
    editIdade = (EditText) findViewById(R.id.txtidade);
    radioSexGroup = (RadioGroup) findViewById(R.id.radioSex);
    btnDisplay = (Button) findViewById(R.id.btnDisplay);
    chkStar = (CheckBox) findViewById(R.id.chkStar);
    chkGame = (CheckBox) findViewById(R.id.chkGame);
    chkSenhor = (CheckBox) findViewById(R.id.chkSenhor);

    //intent = new Intent(this, ResultadoActivity.class);

    btnDisplay.setOnClickListener(new OnClickListener() {
        @Override
        public void onClick(View v) {

            int selectedId = radioSexGroup.getCheckedRadi-
oButtonId();
            radioSexButton = (RadioButton)
findViewById(selectedId);
            cpf = editCpf.getText().toString();
            nome = editNome.getText().toString();
            idade = editIdade.getText().toString();
            sexo = radioSexButton.getText().toString();

            StringBuffer interesses = new StringBuffer();
            if (chkStar.isChecked())
                interesses.append("Star Wars\n");

            if (chkGame.isChecked())
                interesses.append("Game of Thrones\n");

           if (chkSenhor.isChecked())
                interesses.append("Senhor dos Aneis\n");
```

```
                intent.putExtra("sexo", sexo);
                intent.putExtra("nome", nome);
                intent.putExtra("cpf", cpf);
                intent.putExtra("idade", idade);
                intent.putExtra("interesses", interesses.to-
    String());
                //startActivity(intent);
            }
        });
    }
```

Logo abaixo do *layout*, iremos inserir toda a lógica da programação onde permitirá identificar posteriormente os dados digitados e/ou selecionados para envio em outra tela.

Para identificar o sexo selecionado pelo usuário (que poderá ser apenas uma opção, por se tratar de um grupo de *RadioButton*), foi utilizado o comando abaixo:

```
    int selectedId = radioSexGroup.getCheckedRadioButtonId();
```

Neste momento, iremos armazenar, na variável *selectedId,* o número da seleção no qual o usuário selecionou, sendo 0 (zero) para masculino e 1(um) para feminino, conforme ordem estabelecida no *layout*.

Como o objeto do tipo *CheckBox* permite mais de uma seleção, iremos armazenar em um objeto do tipo *StringBuffer* todos os itens selecionados.

```
    StringBuffer interesses = new StringBuffer();
```

Armazenaremos, em variáveis do tipo *String,* os textos digitados em Nome, Cpf e Idade.

Antes de prosseguirmos na explicação, execute seu projeto para verificar a disposição dos objetos criados. Perceba que o mesmo ficou conforme figura abaixo:



Figura 11

O próximo passo será fazer a interação entre telas, na qual levaremos os dados digitados na primeira tela, para uma segunda tela, mas para isso iremos ter que utilizar objeto *Intent*.

Interação entre telas

A classe *android.app.Activity* representa uma tela de aplicação. Uma tela é composta de vários elementos visuais, os quais no Android são representados pela classe *android.view.View*.

O nome *activity_main* é gerado automaticamente pelo Android Studio, mas ele pode ser substituído por outro arquivo. Para cada tela de aplicação, existirá uma *activity* para controlar seu estado e eventos, mas para definir a interface gráfica da tela é utilizada uma *view*. Dentro da *activity* é necessário chamar o método *setContentView(view)* para informar a *view* responsável por desenhar a interface na tela.

A classe R é gerada, automaticamente, pelo Android Studio e contém constantes para acessar os diversos recursos do projeto. Essa classe nunca deve ser alterada manualmente.

A pasta *res* do projeto possui algumas subpastas:

- **drawable** – contém as imagens;
- **layout** – contém os arquivos XML com as telas da aplicação;
- **menu** – contém os arquivos XML com a configuração dos menus;
- **value** – contém os arquivos XML com as mensagens da aplicação.

Sempre que um novo arquivo é adicionado em uma dessas pastas, uma nova constante será criada automaticamente na classe R. Esses arquivos são chamados de recursos.

Cada constante da classe R é criada com o nome da pasta onde o recurso foi inserido e com o mesmo nome do arquivo, sem a extensão. Cada uma dessas constantes é do tipo *int* e contém um número único dentro do projeto.

```
public static final class drawable {
        public static final int ic_launcher=0x7f020000;
 }
```

Independentemente, do tipo do arquivo e da pasta onde ele for adicionado, seu nome deve ser minúsculo e não deve conter espaços. Caso um arquivo de nome inválido for fornecido, a classe R não será compilada corretamente.

O arquivo *AndroidManifest.xml* é a base de uma aplicação Android. **É obrigatório que cada activity do projeto seja declarada no arquivo AndroidManifest.xml.**

Para que a aplicação possa ser iniciada pelo usuário, é necessária pelo menos uma *activity* e esta deve ser configurada como sendo o ponto de partida da aplicação. Para isto, é necessário declarar uma tag *<intent-filter>* com a ação *android.intent.action.Main* e categoria *android.intent.category.LAUNCHER* dentro da *tag <activity>.*

- A ação *MAIN* significa que esta *activity MainActivity* pode ser iniciada isoladamente, como ponto inicial da aplicação. Ela indica que essa *activity* representa um ponto de partida e que não espera nada como entrada.
- A categoria *LAUNCHER* indica que a *activity* estará disponível para o usuário na tela inicial junto com as outras aplicações.

Activity

Uma *Activity* (atividade) é um objeto individual que poderá receber o foco e que poderá ser uma janela a tela inteira (que é o mais frequente), uma janela flutuante ou uma janela embutida dentro de outra janela.

Cada *activity* é responsável por controlar os eventos da tela e definir qual *View* será responsável por desenhar a interface gráfica do usuário. É uma classe que deve herdar da classe *android.app.Activity* ou alguma subclasse desta.

A classe da *Activity* deve implementar o método *onCreate(bundle),* que é obrigatório e responsável por realizar a inicialização necessária para executar a aplicação. Cada activity deve ser declarada no arquivo *AndroidManifest.xml.*

```
    <activity android:name=".teste" />
```

O ponto significa que o pacote da classe é o pacote do projeto. Mas, se necessário, pode-se definir o nome completo da classe.

Estados de uma *activity*:

- executando;
- temporariamente interrompida em segundo plano;
- completamente destruída.

Cada *activity* que é iniciada é inserida no topo de uma pilha (*activity stack*). Assim, sempre que uma nova *activity* é inserida no topo da pilha, a *activity* anterior que estava em execução fica logo abaixo da nova. A *activity* que está no topo da pilha é a *activity* que está em execução no momento, e as demais podem estar executando em segundo plano, estar no estado pausado, ou totalmente paradas.

A figura abaixo mostra o ciclo de vida de uma *activity*, exibindo os estados possíveis e a chamada de cada método.

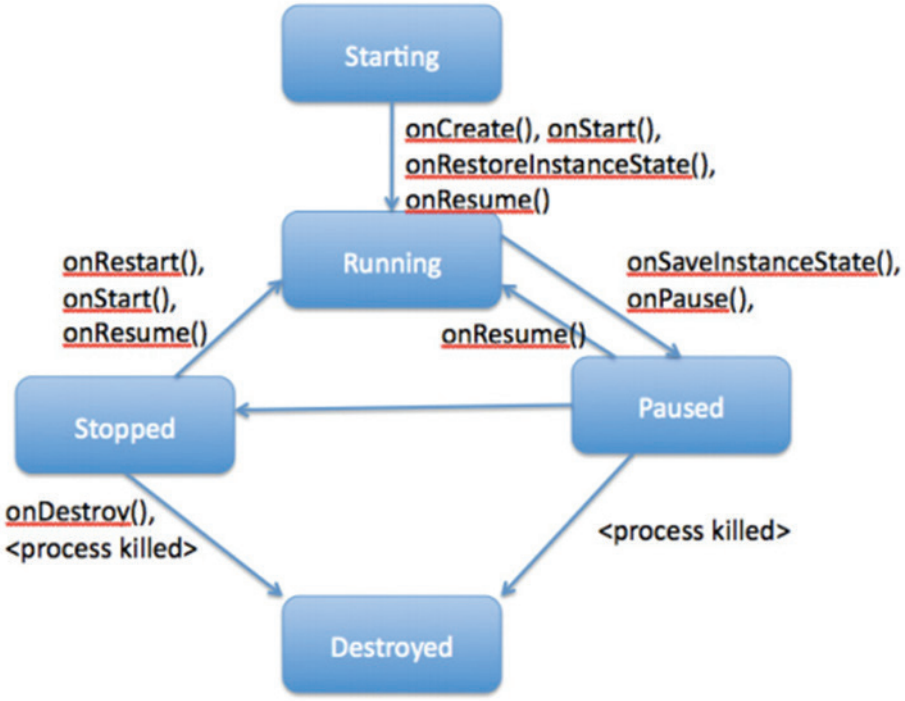


Figura 12

Descrição dos métodos relacionados ao ciclo de vida:

- **onCreate(bundle)** – é obrigatório e chamado uma única vez. Neste método, deve-se criar uma *view* e chamar o método setContentView(view) para exibi-la na tela.
- **onStart()** – é chamado quando a *activity* está ficando visível ao usuário e já tem uma *view*.
- **onRestart()** – é chamado quando uma *activity* foi parada temporariamente e está sendo iniciada outra vez.
- **onResume()** – é chamado quando a *activity* está no topo da pilha e pronta para interagir com o usuário.
- **onPause()** – é chamado quando a *activity* no topo da pilha entra em modo de espera. O método *onResume*() deve ser chamado para que o processo volte a ser executado.
- **onStop()** – quando a *activity* está sendo encerrada e não está mais visível ao usuário.
- **onDestroy()** – é chamado para liberar os recursos da *activity* e removê-la da pilha.

Intent

Podemos considerar que esta classe é a mais poderosa da estrutura do Android. É impossível imaginar a não utilização dela. É por meio dela que a aplicação irá se comunicar, não somente através de janelas da própria aplicação, como também com nas aplicações nativas do Android.

Uma *Intent* (intenção) é a descrição de uma operação a ser realizada e representa uma mensagem da aplicação para o sistema operacional pedindo a execução de algo como

a) abrir outra tela de aplicação;

b) mandar mensagem para outra aplicação;

c) abrir uma página da *Web*;

d) ligar para um número de celular;

e) exibir *Google Maps* etc.

A utilização mais frequente de uma *Intent* é abrir uma *Activity* (navegação entre *Activities*), mas poderá ser utilizada para outras ações como, por exemplo, enviar mensagens ao sistema operacional.

Android fornece o mecanismo para relacionar uma *Intent* com determinada ação que queremos realizar.

Os construtores da classe *Intent*.

Public Constructors
Intent () Create an empty intent.
Intent (Intent o) Copy constructor.
Intent (String action) Create an intent with a given action.
Intent (String action, Uri uri) Create an intent with a given action and for a given data url.
Intent (Context packageContext, Class<?> cls) Create an intent for a specific component.
Intent (String action, Uri uri, Context packageContext, Class<?> cls) Create an intent for a specific component with a specified action and data

Figura 13

Fonte: http://developer.android.com/reference/android/content/Intent.html

Utilizando *Activity* sem esperar resultados

Utilizemos os construtores Intent(Context packageContext, Class<?> cls) e Intent() para criar uma *Intent* e depois abrir uma nova *Activity*.

Primeiramente, iremos criar um novo layout para a nova Activity, no qual receberá os dados que foram digitados na tela de cadastro. Conforme visto anteriormente, crie um novo *layout* com o nome "activity_resultado", e altere o *layout* conforme abaixo:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/
apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    tools:context=".ResultadoActivity" >
    <TextView
        android:id="@+id/textViewcpf"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentLeft="true"
        android:layout_alignParentTop="true"
        android:layout_marginLeft="15dp"
        android:layout_marginTop="20dp"
        android:text="CPF:"
        android:textAppearance="?android:attr/textAppear-
anceLarge" />
    <TextView
        android:id="@+id/tvCPF"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/textView1"
        android:layout_below="@+id/textViewcpf"
        android:text="Medium Text"
        android:textAppearance="?android:attr/textAppearance-
Medium" />
    <TextView
        android:id="@+id/textViewnome"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginLeft="15dp"
        android:layout_marginTop="15dp"
        android:layout_below="@+id/tvCPF"
        android:text="Nome:"
        android:textAppearance="?android:attr/textAppear-
anceLarge" />
```

```
    <TextView
        android:id="@+id/tvNome"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/textView1"
        android:layout_below="@+id/textViewnome"
        android:text="Medium Text"
        android:textAppearance="?android:attr/textAppearance-
Medium" />
    <TextView
        android:id="@+id/textViewidade"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginLeft="15dp"
        android:layout_marginTop="15dp"
        android:layout_below="@+id/tvNome"
        android:text="Idade:"
        android:textAppearance="?android:attr/textAppear-
anceLarge" />
    <TextView
        android:id="@+id/tvIdade"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/textView1"
        android:layout_below="@+id/textViewidade"
        android:text="Medium Text"
        android:textAppearance="?android:attr/textAppearance-
Medium" />
    <TextView
        android:id="@+id/textView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginLeft="15dp"
        android:layout_marginTop="15dp"
        android:layout_below="@+id/tvIdade"
        android:text="Sexo:"
        android:textAppearance="?android:attr/textAppear-
anceLarge" />
    <TextView
        android:id="@+id/textView3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/textView1"
        android:layout_below="@+id/tvSexo"
        android:layout_marginTop="18dp"
        android:text="Interesses:"
        android:textAppearance="?android:attr/textAppear-
anceLarge" />
    <TextView
        android:id="@+id/tvInteresses"
        android:layout_width="wrap_content"
       android:layout_height="wrap_content"
```

```
        android:layout_alignLeft="@+id/textView1"
        android:layout_below="@+id/textView3"
        android:text="Medium Text"
        android:textAppearance="?android:attr/textAppearance-
Medium" />
    <TextView
        android:id="@+id/tvSexo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/textView1"
        android:layout_below="@+id/textView1"
        android:text="Medium Text"
        android:textAppearance="?android:attr/textAppearance-
Medium" />
    <Button
        android:id="@+id/voltar"
        android:text="Voltar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignLeft="@+id/tvSexo"
        android:layout_marginTop="22dp"
        android:layout_below="@+id/tvInteresses"/>
</RelativeLayout>
```

Onde ficará o seguinte layout:

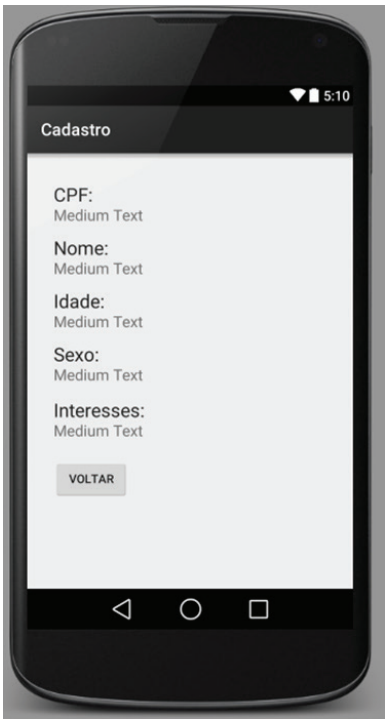


Figura 14

Crie uma nova classe na pasta java com nome **ResultadoActivity.java**

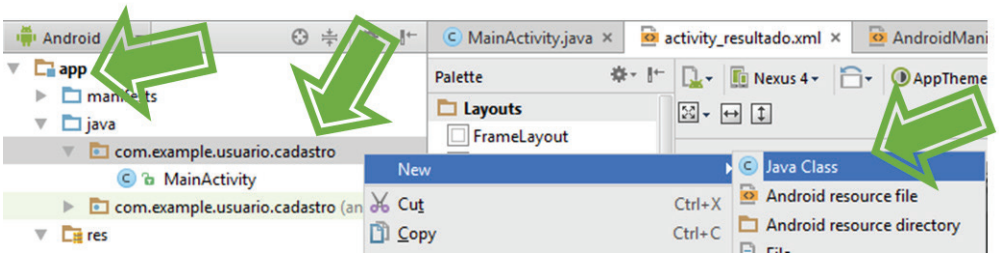


Figura 15

E substitua pelo seguinte código:

```
package com.example.usuario.cadastro;

import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

public class ResultadoActivity extends Activity{

    private TextView tvSexo, tvInteresses,tvCPF,tvNome,tvIda
de;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_resultado);

        Intent intent = getIntent();
        String sexo = intent.getStringExtra("sexo");
        String interesses = intent.
getStringExtra("interesses");
        String cpf = intent.getStringExtra("cpf");
        String nome = intent.getStringExtra("nome");
        String idade = intent.getStringExtra("idade");

        tvSexo = (TextView) findViewById(R.id.tvSexo);
        tvInteresses = (TextView) findViewById(R.
id.tvInteresses);
        tvCPF = (TextView) findViewById(R.id.tvCPF);
        tvNome = (TextView) findViewById(R.id.tvNome);
        tvIdade = (TextView) findViewById(R.id.tvIdade);
       tvSexo.setText(sexo);
        tvInteresses.setText(interesses);
        tvCPF.setText(cpf);
        tvNome.setText(nome);
       tvIdade.setText(idade);

        final Button bvoltar = (Button) findViewById(R.
id.voltar);
```

```
        bvoltar.setOnClickListener(new View.OnClickListener()
{
            public void onClick(View v) {
                finish();
            }
        });
    }
}
```

Agora, iremos programar para que seja ativada uma nova *Activity* (**ResultadoActivity.java**), respondendo ao click no botão **btnDisplay (Mostrar em outra tela)**. Para isso, devemos retirar os comentários na classe *Activity* principal (MainActivity.java), os seguintes comentários. Para retirar os comentários, retire os caracteres "//".

```
intent = new Intent(this, ResultadoActivity.class);

btnDisplay.setOnClickListener(new OnClickListener() {
    @Override
    public void onClick(View v) {

        int selectedId = radioSexGroup.getCheckedRadioButtonId();
        radioSexButton = (RadioButton) findViewById(selectedId);
        cpf = editCpf.getText().toString();
        nome = editNome.getText().toString();
        idade = editIdade.getText().toString();
        sexo = radioSexButton.getText().toString();

        StringBuffer interesses = new StringBuffer();
        if (chkStar.isChecked())
            interesses.append("Star Wars\n");

        if (chkGame.isChecked())
            interesses.append("Game of Thrones\n");

        if (chkSenhor.isChecked())
            interesses.append("Senhor dos Aneis\n");

        intent.putExtra("sexo", sexo);
        intent.putExtra("nome", nome);
        intent.putExtra("cpf", cpf);
        intent.putExtra("idade", idade);
        intent.putExtra("interesses", interesses.toString());
        startActivity(intent);
    }
});
```

Figura 16

Voltando da segunda para a primeira *Activity*

Fechar a segunda *Activity* e voltar para a primeira é bem simples. O método **finish()** da classe **ResultadoActivity** efetua esta ação.

Configuração necessária no arquivo AndroidManifest.xml

Importante!

cada *Activity* (tela) deverá ter uma especificação no arquivo *AndroidManifest.xml* com as tags <*activity*> ... </*activity*>, caso contrário, provocará erro de execução.

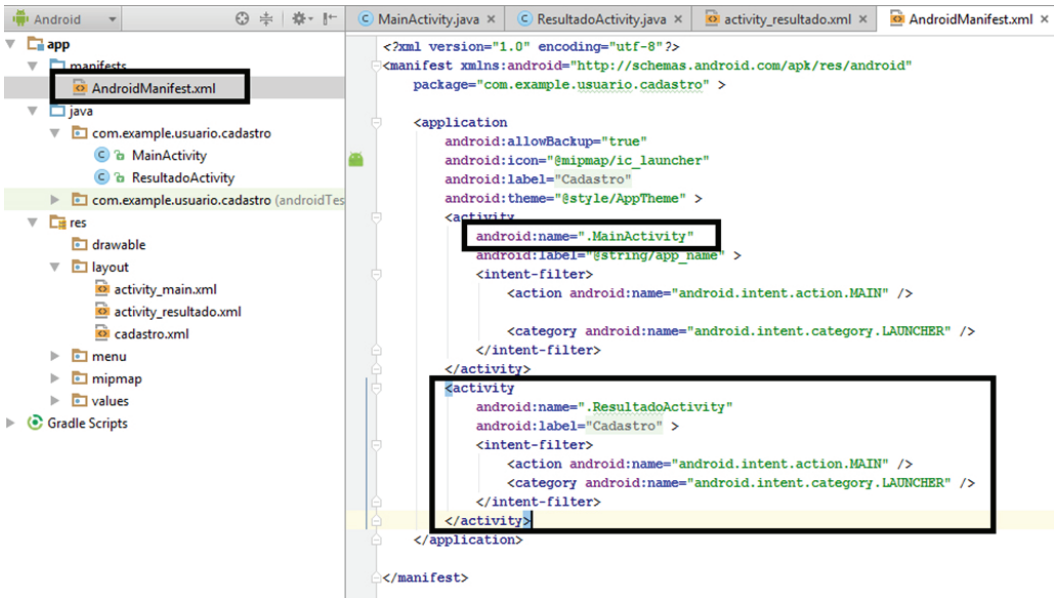


Figura 17

Portanto, dentro do arquivo *AndroidManifest.xml*, dentro da *tag* <*application*, acrescente o seguinte código:

```
<activity
    android:name=".ResultadoActivity"
    android:label="@string/app_name" >
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.
LAUNCHER" />
    </intent-filter>
</activity>
```

Explicação

A classe MainActivity, o método btnDisplay.setOnClickListener(new OnClickListener() recupera as informações que o usuário digitou e selecionou na tela e serão enviados posteriormente para a Activity destino (ResultadoActivity) através do método putExtra.

```
intent.putExtra("sexo", sexo);
intent.putExtra("nome", nome);
intent.putExtra("cpf", cpf);
intent.putExtra("idade", idade);
intent.putExtra("interesses", interesses.toString());
startActivity(intent);
```

Figura 18

Os nomes que estão entre aspas serão recuperados na Activity de destino com os valores que foram gravados nela. Perceba que o método startActivity(itent) irá chamar a Activity de destino (ResultadoActivity).

Na activity de destino, perceba que será executado primeiramente o método onCreate onde este tornará ativo o layout activity_resultado como ativo.

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_resultado);

    Intent intent = getIntent();
    String sexo = intent.getStringExtra("sexo");
    String interesses = intent.getStringExtra("interesses");
    String cpf = intent.getStringExtra("cpf");
    String nome = intent.getStringExtra("nome");
    String idade = intent.getStringExtra("idade");

    tvSexo = (TextView) findViewById(R.id.tvSexo);
    tvInteresses = (TextView) findViewById(R.id.tvInteresses);
    tvCPF = (TextView) findViewById(R.id.tvCPF);
    tvNome = (TextView) findViewById(R.id.tvNome);
    tvIdade = (TextView) findViewById(R.id.tvIdade);

    tvSexo.setText(sexo);
    tvInteresses.setText(interesses);
    tvCPF.setText(cpf);
    tvNome.setText(nome);
    tvIdade.setText(idade);

    final Button bvoltar = (Button) findViewById(R.id.voltar);
    bvoltar.setOnClickListener(new View.OnClickListener() {
        public void onClick(View v) {
            finish();
        }
    });
}
```

Figura 19

Logo abaixo do ativar o layout, serão gravados em variáveis os valores nos quais foram enviados pela Activity principal (MainActivity) através do método putExtra.

```
Intent intent = getIntent();
String sexo = intent.getStringExtra("sexo");
String interesses = intent.getStringExtra("interesses");
String cpf = intent.getStringExtra("cpf");
String nome = intent.getStringExtra("nome");
String idade = intent.getStringExtra("idade");
```

Figura 20

Logo abaixo são referenciadas as variáveis com os objetos declarados no layout e atribuídos a eles os valores nos quais foram recuperados nas variáveis acima.

```
tvSexo = (TextView) findViewById(R.id.tvSexo);
tvInteresses = (TextView) findViewById(R.id.tvInteresses);
tvCPF = (TextView) findViewById(R.id.tvCPF);
tvNome = (TextView) findViewById(R.id.tvNome);
tvIdade = (TextView) findViewById(R.id.tvIdade);

tvSexo.setText(sexo);
tvInteresses.setText(interesses);
tvCPF.setText(cpf);
tvNome.setText(nome);
tvIdade.setText(idade);
```

Figura 21

Depois, é programado o botão voltar, o qual irá fechar a Activity em questão.

```
final Button bvoltar = (Button) findViewById(R.id.voltar);
bvoltar.setOnClickListener(new View.OnClickListener() {
    public void onClick(View v) {
        finish();
    }
});
```

Figura 22

Material Complementar

Para aprofundar seus estudos sobre View, eventos, activity e intente, consulte os links abaixo:

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Sites

Android - Ciclo de vida de uma atividade
https://goo.gl/JHm4L2

GUJ
http://www.guj.com.br/

Vídeos

Android Tutorial #7: Create & launch an activity using explicit intent from a menu item
https://youtu.be/JBSfpEsT4jk

Referências

ANDROID. **Android Developers**. Disponível em http://developer.android.com

LECHETA, R. R. **Google Android, Aprenda a criar aplicações para dispositivos móveis com o Android SDK**. São Paulo: Novatec, 2010.

MEDNIEKS, Z. et. al. **Desenvolvimento de Aplicações Android**. São Paulo: Novatec, 2009.

LEE, W. M. **Introdução ao Desenvolvimento de Aplicativos para o Android**. Rio de Janeiro: Ciência Moderna, 2011.

Cruzeiro do Sul Virtual
Educação a Distância

www.cruzeirodosulvirtual.com.br
Campus Liberdade
Rua Galvão Bueno, 868
CEP 01506-000
São Paulo - SP - Brasil
Tel: (55 11) 3385-3000


Cruzeiro do Sul
Educacional